FISCALIZA

Documentação Técnica Completa

Aplicativo Mobile de Fiscalização Ambiental de Madeireiras

Versão 1.0 | 2025

Plataforma: Flutter (iOS & Android) | Offline-First

Trabalho de Conclusão de Curso — Documentação de Sistema

Sumário

1. Visão Geral do Sistema

1.1 Descrição

O FISCALIZA é um aplicativo mobile desenvolvido em Flutter, voltado para fiscais ambientais que realizam inspeções em madeireiras. Seu objetivo é digitalizar e automatizar o processo de contagem, classificação e validação de produtos de madeira, cruzando os dados coletados em campo com as informações declaradas no Documento de Origem Florestal (DOF).

O sistema opera 100% offline, sem dependência de conectividade de rede durante a fiscalização. Toda a inteligência de visão computacional é executada localmente no dispositivo, utilizando um modelo YOLO convertido para TensorFlow Lite.

1.2 Objetivos

- Digitalizar o processo de fiscalização de madeireiras eliminando registros em papel
- Automatizar a contagem de unidades de madeira por meio de visão computacional
- Validar os dados fiscalizados contra as declarações do DOF
- Gerar relatórios estruturados ao final de cada fiscalização
- Funcionar integralmente offline em campo

1.3 Escopo e Limitações

Dentro do Escopo

Upload e leitura de planilha DOF (Excel), detecção de madeiras por YOLO/TFLite, contagem manual corretiva, cadastro de espécies e dimensões, cálculo de volume (m^3), comparação com DOF, geração de relatório em PDF.

Fora do Escopo (v1.0)

Integração com APIs governamentais online, autenticação de usuário com servidor remoto, sincronização em nuvem em tempo real, classificação automática de espécies por IA.

2. Arquitetura do Sistema

2.1 Arquitetura Geral

O aplicativo adota Clean Architecture combinada com o padrão MVVM (Model-View-ViewModel), dividindo o sistema em camadas bem definidas que garantem separação de responsabilidades, testabilidade e manutenibilidade.

A separação em camadas segue o seguinte fluxo de dependências: a camada de Presentation depende da Domain, a camada de Data depende da Domain, e a Domain não depende de nenhuma outra camada interna.

2.2 Camadas da Arquitetura

Camada de Presentation (UI)

Responsável por toda a interface com o usuário. Contém Screens (páginas Flutter), Widgets reutilizáveis e ViewModels/Controllers (gerenciamento de estado com Riverpod). Não contém lógica de negócio.

Camada de Domain

Núcleo da aplicação. Contém as Entidades de negócio, os Repositórios (interfaces abstratas), os Use Cases (regras de negócio isoladas) e os modelos de domínio. É totalmente independente de frameworks externos.

Camada de Data

Implementa as interfaces definidas no Domain. Contém os Repositórios concretos, Data Sources locais (Isar/SQLite, sistema de arquivos), Modelos de dados com serialização e os serviços de integração (TFLite, leitura de Excel, geração de XML/PDF).

2.3 Estrutura de Pastas do Projeto

```
lib/
├── core/
│   ├── constants/          # Constantes globais (limites, margens)
│   ├── errors/             # Classes de exceção customizadas
│   ├── utils/              # Utilitários (formatação, cálculos)
│   └── theme/              # Tema visual do app
│
├── features/
│   ├── dof/                # Feature: Upload e leitura do DOF
│   │   ├── data/
│   │   │   ├── datasources/
│   │   │   │   └── dof_local_datasource.dart
```

```
│   │   │   ├── models/
│   │   │   │   └── dof_item_model.dart
│   │   │   └── repositories/
│   │   │       └── dof_repository_impl.dart
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   │   └── dof_item.dart
│   │   │   ├── repositories/
│   │   │   │   └── dof_repository.dart
│   │   │   └── usecases/
│   │   │       ├── load_dof_from_excel.dart
│   │   │       └── export_dof_to_xml.dart
│   │   └── presentation/
│   │       ├── screens/
│   │       ├── widgets/
│   │       └── viewmodels/
│   │
│   ├── fiscalizacao/       # Feature: Captura e validação
│   │   ├── data/ ...
│   │   ├── domain/ ...
│   │   └── presentation/ ...
│   │
│   ├── resultado/          # Feature: Cálculo e comparação
│   └── relatorio/          # Feature: Geração de relatório
│
├── shared/
│   ├── widgets/            # Widgets compartilhados
│   └── providers/          # Providers globais (Riverpod)
│
└── main.dart
```

```
assets/
├── models/                 # Arquivos .tflite e labels.txt
└── icons/
```

2.4 Gerenciamento de Estado

O estado da aplicação é gerenciado com Riverpod (flutter_riverpod). A escolha se justifica pela tipagem segura em tempo de compilação, suporte nativo a operações assíncronas (AsyncNotifier), ausência de BuildContext nos providers e facilidade de teste unitário.

Tipo de Provider	Uso no App	Exemplo
StateNotifierProvider	Estado mutável e complexo	FiscalizacaoViewModel
FutureProvider	Dados assíncronos de leitura	dofItemsProvider
Provider	Instâncias de repositórios	dofRepositoryProvider
AsyncNotifierProvider	Operações com loading/error	captureNotifierProvider

3. Tecnologias Recomendadas

3.1 Framework Principal

Flutter (Dart) é o framework escolhido por permitir desenvolvimento multiplataforma (iOS e Android) a partir de uma única base de código, com compilação nativa para alto desempenho — fator crítico para execução de modelos de ML localmente.

3.2 Execução do Modelo de Visão Computacional

TensorFlow Lite (Recomendado)

O TensorFlow Lite é a escolha primária para execução do modelo YOLO no dispositivo. Justificativas:

- Suporte nativo a aceleração por hardware via GPU Delegate e NNAPI (Android) / Core ML Delegate (iOS)
- Integração direta com o plugin tflite_flutter do Flutter
- Ecossistema maduro com ampla documentação
- Compatível com modelos YOLOv5 e YOLOv8 exportados para .tflite

Alternativas Avaliadas

Tecnologia	Vantagem	Desvantagem	Decisão
TensorFlow Lite	Melhor suporte Flutter, GPU delegate	Conversão YOLO necessária	ADOTADO
ONNX Runtime	Portabilidade entre frameworks	Suporte Flutter ainda limitado	Alternativa futura
MediaPipe	Pipelines prontos para visão	Não suporta YOLO customizado	Descartado
PyTorch Mobile	Fiel ao YOLO original	Binding Flutter inexistente	Descartado

3.3 Armazenamento Local

Isar Database (Recomendado)

Isar é um banco de dados NoSQL de alta performance para Flutter, com suporte a consultas reativas, schemas tipados em Dart e transações ACID. É significativamente mais rápido que SQLite para operações de leitura/escrita de objetos complexos e possui integração nativa com Riverpod.

Armazenamento de Arquivos

Imagens e o arquivo XML do DOF são armazenados no sistema de arquivos local do dispositivo, acessados via path_provider. O caminho de armazenamento usa o diretório de documentos do aplicativo, persistindo entre sessões.

4. Bibliotecas Flutter

4.1 Tabela de Dependências

Biblioteca	Versão	Finalidade	Camada
flutter_riverpod	^2.5.x	Gerenciamento de estado	Presentation
isar / isar_flutter_libs	^3.1.x	Banco de dados local	Data
tflite_flutter	^0.10.x	Inferência TFLite no dispositivo	Data/Service
image	^4.1.x	Processamento e manipulação de imagens	Data/Service
camera	^0.10.x	Acesso à câmera nativa	Data
excel	^4.0.x	Leitura de arquivos .xlsx/.xls	Data
xml	^6.5.x	Geração e manipulação de XML	Data
file_picker	^8.0.x	Seleção de arquivos pelo usuário	Presentation
path_provider	^2.1.x	Acesso a diretórios do sistema	Data
pdf / printing	^3.11.x	Geração de relatórios PDF	Data/Service
go_router	^13.x	Navegação declarativa	Presentation
freezed	^2.5.x	Imutabilidade e copyWith em entidades	Domain
json_serializable	^6.7.x	Serialização de modelos JSON	Data
flutter_svg	^2.0.x	Ícones e ilustrações vetoriais	Presentation
permission_handler	^11.x	Permissões de câmera e armazenamento	Data
share_plus	^9.x	Compartilhamento do relatório gerado	Presentation

4.2 Observações sobre Bibliotecas Críticas

tflite_flutter

Esta é a biblioteca central para execução do modelo. Requer configuração adicional no Android (build.gradle) para incluir as bibliotecas nativas .so do TensorFlow Lite e ativar o GPU Delegate.

- Android: adicionar abiFilters e pickFirst para libs nativas
- iOS: adicionar TensorFlowLiteC via CocoaPods

- Testar em dispositivo físico — emuladores não suportam GPU Delegate

excel

Lê arquivos .xlsx sem dependência nativa, funcionando em puro Dart. Suporta múltiplas planilhas, tipos de dados (String, double, int, DateTime) e formatação de células. A leitura deve ser realizada em Isolate separado para não bloquear a UI.

5. Integração com YOLO e TensorFlow Lite

5.1 Pipeline de Conversão do Modelo

O modelo YOLO precisa ser treinado previamente com imagens de madeiras empilhadas e, em seguida, convertido para o formato .tflite. O fluxo completo de conversão é:

1. Treinamento com YOLOv8 (Python/PyTorch)

Treinar o modelo com dataset de madeiras rotuladas. Exportar como PyTorch (.pt).

2. Exportação para TFLite

Usar o utilitário oficial do Ultralytics para exportar direto para TFLite quantizado (INT8):

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.export(format='tflite', int8=True, imgsz=640)
```

3. Arquivos resultantes

O processo gera: best_float32.tflite (FP32), best_int8.tflite (quantizado INT8 — recomendado para mobile) e labels.txt com as classes.

4. Inclusão no app Flutter

Copiar os arquivos .tflite e labels.txt para assets/models/ e declarar no pubspec.yaml.

5.2 Pré-processamento da Imagem

Antes de enviar ao modelo, a imagem capturada pela câmera deve ser normalizada:

- Redimensionar para 640x640 pixels (ou tamanho de entrada do modelo)
- Converter de BGRA/YUV420 (formato câmera) para RGB
- Normalizar pixels para o intervalo [0.0, 1.0] (Float32) ou manter em uint8 (INT8)
- Criar tensor de shape [1, 640, 640, 3]

Esta operação deve ser realizada em um Isolate do Dart ou usando a biblioteca image para não travar a UI.

5.3 Pós-processamento das Saídas

O modelo YOLOv8 TFLite retorna um tensor de saída com shape [1, 84, 8400] (para 80 classes + 4 bbox coords). O pós-processamento envolve:

- Transpor o tensor para [8400, 84]
- Extrair coordenadas (cx, cy, w, h) e scores de cada classe para cada uma das 8400 detecções
- Filtrar detecções com score abaixo do threshold (padrão: 0.40)
- Aplicar Non-Maximum Suppression (NMS) com IoU threshold de 0.45 para remover caixas duplicadas

- Converter coordenadas de centro para (x1, y1, x2, y2) em pixels relativos

O resultado final é uma lista de objetos Detection com campos: boundingBox (Rect), score (double) e label (String).

5.4 Renderização dos Bounding Boxes

Os bounding boxes são renderizados sobre a imagem usando um CustomPainter Flutter. O painter itera sobre a lista de detecções, desenhando retângulos coloridos (Canvas.drawRect) e rótulos de texto (TextPainter). A posição é calculada proporcionalmente ao tamanho do widget de exibição versus o tamanho original da imagem.

5.5 Treinamento do Dataset

Considerações para o Dataset de Treinamento

O modelo precisa ser treinado com imagens reais de madeiras empilhadas em madeireiras, capturadas em diferentes condições de iluminação, ângulos e espécies. Recomenda-se um mínimo de 500 a 1000 imagens anotadas (bounding boxes por unidade de madeira) para obter resultados satisfatórios. Ferramentas como Roboflow ou Label Studio podem ser usadas para anotação e augmentation do dataset.

6. Fluxo de Telas

6.1 Mapa de Navegação

O app utiliza go_router para navegação declarativa. O fluxo principal é linear, mas permite retorno e correções a qualquer momento antes da finalização.

Rota	Tela	Descrição
/	SplashScreen	Inicialização, carregamento de providers e redirect
/upload-dof	UploadDofScreen	Upload e processamento da planilha DOF
/fiscalizacao	FiscalizacaoScreen	Hub central — lista produtos a fiscalizar
/fiscalizacao/captura	CapturaScreen	Câmera, detecção e contagem por foto
/fiscalizacao/validacao	ValidacaoScreen	Confirmação manual da contagem
/fiscalizacao/cadastro	CadastroScreen	Espécie, dimensões e variações
/resultado	ResultadoScreen	Comparação com DOF e classificação
/relatorio	RelatorioScreen	Visualização e exportação do relatório PDF

6.2 Tela 1 — Upload do DOF

Campos e componentes presentes na tela:

- Área de arrastar e soltar (ou botão) para seleção do arquivo Excel via file_picker
- Indicador de progresso durante o parsing da planilha
- Tabela de prévia mostrando os itens lidos: Número, Produto, Espécie, Nome Popular, Saldo Livre, Saldo Total, Unidade
- Botão de confirmação para prosseguir — desabilitado até leitura bem-sucedida
- Mensagem de erro em caso de formato inválido

Ao confirmar, os dados são convertidos para XML e salvos localmente via Isar, e o app navega para /fiscalizacao.

6.3 Tela 2 — Hub de Fiscalização

Exibe a lista de produtos declarados no DOF. Cada item mostra: nome do produto, espécie, saldo declarado, status (Pendente / Em andamento / Concluído / Excedente). Um FAB (+) permite iniciar a fiscalização de um produto. Ao final, é possível registrar produtos extras não presentes no DOF.

6.4 Tela 3 — Captura de Imagem

É a tela de maior complexidade técnica. Fluxo dentro da tela:

5. O fiscal visualiza o preview da câmera em tempo real
6. Pressiona o botão de captura
7. A imagem é capturada e enviada para processamento (com indicador de loading)
8. A imagem é exibida com bounding boxes sobrepostos
9. Abaixo da imagem: quantidade detectada, média de confiança e lista de detecções
10. Se confiança média < threshold configurado (ex: 60%), exibir alerta sugerindo nova captura
11. Botão 'Confirmar esta foto' avança para ValidacaoScreen
12. Botão 'Tirar nova foto' descarta e retorna ao preview

É permitido acumular múltiplas fotos para um mesmo produto antes de prosseguir.

6.5 Tela 4 — Validação Manual

Exibe o total detectado automaticamente (soma de todas as fotos). O fiscal pode:

- Confirmar o valor automaticamente detectado
- Incrementar manualmente (botões + e -) para corrigir madeiras não detectadas
- Informar o número de madeiras não detectadas por foto individualmente

O valor final confirmado é armazenado como contagem_confirmada, distinto da contagem_automatica.

6.6 Tela 5 — Cadastro de Dados

Formulário preenchido após confirmação da contagem:

- Seletor de espécie (dropdown com espécies do DOF carregadas do Isar)
- Campos de dimensão principal: largura (cm), altura (cm), comprimento (m)
- Checkbox/switch 'Há madeiras com dimensão diferente neste lote?'
- Se ativado: campo para quantidade com dimensão alternativa + campos de dimensão alternativa
- Botão Salvar — persiste o registro de fiscalização completo no Isar

6.7 Tela 6 — Resultado da Fiscalização

Após fiscalizar todos os itens obrigatórios do DOF, esta tela é acessível:

- Cards por produto com: volume declarado (m^3), volume fiscalizado (m^3), diferença absoluta e percentual
- Indicador visual colorido: verde (dentro da margem), amarelo (leve desvio), vermelho (acima de 10%)
- Resumo global da fiscalização
- Botão para navegar ao relatório

6.8 Tela 7 — Relatório

Exibição da prévia do relatório e opções de exportação:

- Prévia do documento PDF renderizado in-app
- Botão Exportar PDF — salva no armazenamento do dispositivo
- Botão Compartilhar — abre seletor de apps via share_plus

7. Regras de Negócio

7.1 Detecção e Contagem

Regra	Descrição	Valor Padrão
Threshold de confiança	Score mínimo para uma detecção ser considerada válida	0.40 (40%)
Threshold de alerta	Média de confiança abaixo deste valor gera sugestão de nova foto	0.60 (60%)
IoU para NMS	Limiar para Non-Maximum Suppression (eliminar duplicatas)	0.45
Máximo de fotos por produto	Limite de imagens acumuladas antes de confirmar	Sem limite (configurável)
Contagem mínima	Fiscalização só pode ser salva com pelo menos 1 madeira contada	1 unidade

7.2 Cálculo de Volume (m^3)

O volume em metros cúbicos de cada lote fiscalizado é calculado pela fórmula:

Fórmula de Cálculo de Volume

Volume (m^3) = (Largura (m) × Altura (m) × Comprimento (m)) × QuantidadePara lotes com madeiras de dimensões diferentes:Volume Total = ($Largura_1$ × $Altura_1$ × $Comprimento_1$ × Qtd_1) + ($Largura_2$ × $Altura_2$ × $Comprimento_2$ × Qtd_2)

As dimensões informadas pelo fiscal em centímetros são convertidas para metros antes do cálculo. Comprimento já deve ser informado em metros.

7.3 Comparação com o DOF

Classificação	Critério	Indicador Visual
Dentro do esperado	Volume fiscalizado está dentro de ±10% do declarado	Verde
Acima da margem	Volume fiscalizado > volume declarado × 1.10	Vermelho
Abaixo da margem	Volume fiscalizado < volume declarado × 0.90	Amarelo
Não declarado	Produto fiscalizado não consta no DOF	Roxo/Alerta

7.4 Correção Manual

- A correção manual sempre prevalece sobre a contagem automática
- O sistema armazena tanto a contagem automática quanto a confirmada para fins de auditoria
- O fiscal pode indicar quantas madeiras foram não detectadas (valor incremental, não substitutivo)
- Não é possível informar uma contagem confirmada menor que zero

7.5 Obrigatoriedade da Fiscalização

- Todos os itens do DOF devem ser fiscalizados antes que o resultado geral seja liberado
- O fiscal pode adicionar produtos extras (não constantes no DOF) ao final
- Uma fiscalização pode ser salva parcialmente e retomada em outra sessão

8. Modelagem de Dados

8.1 Entidades Principais

DofItem — Item do Documento de Origem Florestal

Campo	Tipo	Descrição
id	String (UUID)	Identificador único gerado localmente
numero	String	Número do item no DOF original
produto	String	Nome do produto madeireiro
especieCientifico	String	Nome científico da espécie
nomePopular	String	Nome popular da espécie
saldoLivre	double	Volume disponível para movimentação (m^3)
saldoTotal	double	Volume total declarado (m^3)
unidade	String	Unidade de medida (padrão: m^3)

SessaoFiscalizacao — Sessão de Fiscalização

Campo	Tipo	Descrição
id	String (UUID)	Identificador único
dataInicio	DateTime	Timestamp de início da fiscalização
dataFim	DateTime?	Timestamp de conclusão (null se em andamento)
xmlDofPath	String	Caminho para o arquivo XML do DOF
status	Enum	RASCUNHO | EM_ANDAMENTO | CONCLUIDA
observacoes	String?	Campo livre para observações do fiscal

RegistroFiscalizacao — Registro por Produto

Campo	Tipo	Descrição
id	String (UUID)	Identificador único
sessaoId	String	FK para SessaoFiscalizacao
dofItemId	String?	FK para DofItem (null se produto extra)
contagemAutomatica	int	Total de detecções pelo modelo YOLO
contagemConfirmada	int	Total confirmado/corrigido pelo fiscal

Campo	Tipo	Descrição
volumeCalculado	double	Volume total em m³ calculado
fotos	List<FotoFiscalizacao>	Lista de fotos associadas
dimensoes	List<DimensaoLote>	Dimensões informadas pelo fiscal
produtoExtra	bool	true se produto não consta no DOF

FotoFiscalizacao — Foto Individual

Campo	Tipo	Descrição
id	String (UUID)	Identificador único
registroId	String	FK para RegistroFiscalizacao
caminhoArquivo	String	Caminho absoluto da imagem no dispositivo
contagemAutomatica	int	Detecções nesta foto específica
contagemCorrecao	int	Madeiras adicionadas manualmente nesta foto
mediaConfianca	double	Score médio das detecções (0.0 a 1.0)
bboxJson	String	Serialização JSON dos bounding boxes
timestamp	DateTime	Data e hora da captura

DimensaoLote — Dimensões de um Lote

Campo	Tipo	Descrição
id	String (UUID)	Identificador único
registroId	String	FK para RegistroFiscalizacao
larguraCm	double	Largura em centímetros
alturaCm	double	Altura em centímetros
comprimentoM	double	Comprimento em metros
quantidade	int	Quantidade de madeiras com esta dimensão

ResultadoFiscalizacao — Resultado Final por Produto

Campo	Tipo	Descrição
dofItemId	String?	FK para DofItem
volumeDeclarado	double	Volume do DOF (m³)
volumeFiscalizado	double	Volume medido em campo (m³)
diferencaAbsoluta	double	volumeFiscalizado - volumeDeclarado

Campo	Tipo	Descrição
diferencePercentual	double	Diferença em % em relação ao declarado
classificacao	Enum	CONFORME | ACIMA | ABAIXO | NAO_DECLARADO

8.2 Estrutura do XML do DOF

O XML gerado a partir da planilha segue a estrutura:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<dof>
  <metadata>
    <dataImportacao>2025-08-01T14:32:00</dataImportacao>
    <totalItens>12</totalItens>
  </metadata>
  <itens>
    <item>
      <numero>001</numero>
      <produto>Tora de Madeira</produto>
      <especieCientifico>Swietenia macrophylla</especieCientifico>
      <nomePopular>Mogno</nomePopular>
      <saldoLivre>45.80</saldoLivre>
      <saldoTotal>100.00</saldoTotal>
      <unidade>m3</unidade>
    </item>
  </itens>
</dof>
```

9. Geração de Relatório

9.1 Conteúdo do Relatório

O relatório PDF é gerado com a biblioteca pdf (dart_pdf) e renderizado com o widget printing. O documento contém:

- Cabeçalho: dados do fiscal, data/hora da fiscalização, identificação do DOF
- Resumo executivo: número de produtos fiscalizados, situação geral (conforme/irregular)
- Tabela detalhada por produto: declarado vs. fiscalizado, diferença, classificação
- Seção de evidências: imagens capturadas com bounding boxes, organizadas por produto
- Produtos extras: itens encontrados não declarados no DOF
- Rodapé com número de página e timestamp

9.2 Exportação e Compartilhamento

- O PDF é salvo no diretório de downloads do dispositivo via path_provider
- O botão Compartilhar usa share_plus para abrir o seletor nativo de apps
- Formatos de saída: PDF (único formato suportado na v1.0)

10. Desafios Técnicos e Mitigações

10.1 Tabela de Desafios

Desafio	Impacto	Estratégia de Mitigação
Performance do modelo em devices de baixo custo	Alto	Quantização INT8; model size < 5MB; processar em Isolate; usar GPU Delegate quando disponível
Qualidade variável das fotos de campo	Alto	Threshold adaptativo; sugestão de nova captura; Flash automático; orientação ao fiscal
Oclusão e sobreposição de madeiras	Alto	Augmentation de dataset com imagens ocluídas; ajuste de IoU threshold; correção manual sempre disponível
Conversão de planilha com formato variável	Médio	Detecção automática de cabeçalhos; validação de campos obrigatórios; feedback de erro específico
Armazenamento de imagens (espaço em disco)	Médio	Compressão JPEG (qualidade 80%); limpeza automática de sessões antigas
Geração de PDF com imagens grandes	Médio	Redimensionar imagens para relatório (max 800px); gerar em background Isolate
Precisão do cálculo de volume	Médio	Usar double com precisão controlada; arredondar para 4 casas decimais

11. Configurações e Constantes do Sistema

11.1 Constantes Configuráveis

```
// lib/core/constants/app_constants.dart

class AppConstants {
  // Modelo YOLO
  static const String modelPath = 'assets/models/yolo_madeira.tflite';
  static const String labelsPath = 'assets/models/labels.txt';
  static const int modelInputSize = 640;
  static const double confidenceThreshold = 0.40;
  static const double alertConfidenceThreshold = 0.60;
  static const double iouThreshold = 0.45;

  // Regras de negócio
  static const double margemTolerancia = 0.10; // 10%

  // Armazenamento
  static const int maxImageQuality = 80;
  static const int maxImageWidth = 1920;
}
```

12. Melhorias Futuras (Roadmap)

12.1 Versão 1.1 — Melhorias Operacionais

- Geolocalização: registrar coordenadas GPS de cada foto e da fiscalização completa
- Assinatura digital: campo para assinatura do fiscal e do responsável da madeireira
- Suporte a múltiplos fiscais: login local com perfis distintos
- Histórico de fiscalizações: visualização e busca de sessões passadas

12.2 Versão 2.0 — IA e Automação

- Classificação automática de espécies: modelo CNN separado para identificar a espécie pela textura da madeira
- Detecção de anomalias: identificar madeiras com cortes irregulares ou de origem suspeita
- Melhorias no dataset: aprendizado federado com dados de múltiplos fiscais

12.3 Versão 3.0 — Integração Sistêmica

- API DOF: integração direta com o sistema do IBAMA para importação automática do DOF
- Sincronização em nuvem: upload automático quando conectado ao Wi-Fi
- Dashboard web: painel para gestores visualizarem fiscalizações em tempo real
- Integração com sistemas estaduais de controle florestal

12.4 Considerações de Segurança Futuras

- Criptografia dos dados locais armazenados no Isar
- Autenticação biométrica (fingerprint/face ID) para acesso ao app
- Integridade das imagens: hash SHA-256 para detectar adulteração
- Log de auditoria imutável de todas as ações realizadas

13. Ambiente de Desenvolvimento

13.1 Requisitos

Ferramenta	Versão Mínima	Observação
Flutter SDK	3.19.x	Canal stable recomendado
Dart SDK	3.3.x	Incluído no Flutter SDK
Android Studio / VS Code	Última estável	Com plugins Flutter e Dart
Android SDK	API 24 (Android 7.0)	Versão mínima de suporte
iOS Deployment Target	iOS 13.0	Necessário para tflite_flutter
Python	3.9+	Para conversão do modelo YOLO
Ultralytics	8.x	pip install ultralytics

13.2 Configuração do Android para TFLite

```
// android/app/build.gradle
android {
  defaultConfig {
    ndk {
      abiFilters 'armeabi-v7a', 'arm64-v8a', 'x86_64'
    }
  }
  packagingOptions {
    pickFirst '**/*.so'
  }
}
```

13.3 pubspec.yaml — Seção de Assets

```
flutter:
  assets:
    - assets/models/yolo_madeira.tflite
    - assets/models/labels.txt
    - assets/icons/
```

14. Considerações Acadêmicas (TCC)

14.1 Justificativa Tecnológica

A escolha do Flutter se justifica academicamente pela sua capacidade de gerar aplicações nativas para múltiplas plataformas a partir de uma única codebase em Dart, reduzindo o esforço de desenvolvimento e o escopo do trabalho. O framework é sustentado pela Google e possui uma comunidade crescente.

A adoção do TensorFlow Lite para inferência local se alinha com a tendência de Edge AI — processamento de inteligência artificial diretamente no dispositivo, sem envio de dados sensíveis a servidores externos. Isso é especialmente relevante em contextos de fiscalização ambiental, onde a privacidade dos dados e a disponibilidade offline são requisitos fundamentais.

14.2 Contribuições Científicas Potenciais

- Avaliação de precisão de modelos YOLO para detecção de objetos cilíndricos empilhados em contexto ambiental
- Comparação entre estratégias de quantização (FP32 vs INT8) para trade-off entre precisão e velocidade em dispositivos Android de faixa de entrada
- Proposta de workflow de fiscalização ambiental digital com validação humana no loop

14.3 Métricas de Avaliação Sugeridas

Métrica	Como medir	Meta
Precisão da contagem	Comparar contagem automática vs. contagem real por um especialista em N amostras	> 85% de acerto
Tempo de inferência	Medir tempo médio de processamento por imagem no dispositivo-alvo	< 3 segundos
Usabilidade	Escala SUS (System Usability Scale) com fiscais reais	> 70 pontos
Exatidão do volume	Comparar m³ calculado vs. medição manual controlada	Desvio < 5%

15. Glossário

Termo	Definição
DOF	Documento de Origem Florestal — documento obrigatório para transporte de produtos florestais no Brasil, emitido pelo IBAMA
YOLO	You Only Look Once — família de modelos de detecção de objetos em tempo real baseados em redes neurais convolucionais
TFLite	TensorFlow Lite — versão otimizada do TensorFlow para dispositivos móveis e embarcados
Bounding Box	Retângulo delimitador desenhado ao redor de um objeto detectado pelo modelo de visão computacional
NMS	Non-Maximum Suppression — técnica para eliminar detecções duplicadas, mantendo apenas a de maior confiança por região
IoU	Intersection over Union — métrica que mede o grau de sobreposição entre dois bounding boxes
m^3	Metro cúbico — unidade de medida volumétrica utilizada para quantificar produtos madeireiros
INT8	Quantização de 8 bits inteiros — técnica para reduzir o tamanho e aumentar a velocidade de modelos de ML com pequena perda de precisão
MVVM	Model-View-ViewModel — padrão arquitetural que separa a lógica de apresentação (ViewModel) da interface (View)
Clean Architecture	Arquitetura em camadas proposta por Robert C. Martin que isola regras de negócio de detalhes de implementação
Isolate	Mecanismo de concorrência do Dart — equivalente a threads, mas sem memória compartilhada
Riverpod	Biblioteca de gerenciamento de estado para Flutter, evolução do Provider com tipagem mais segura